

Contact Management System

Internship Project Report

Submitted By,

M. Bhavya Sai Sree

Under the Guidance of

Jayanth Varma Konduru



Block – B, 2nd Floor, Phoenix Primea,
40 & 41, Road No. 2, Financial District,
Nanakramguda, Hyderabad, Telangana – 500032.

Index

Content	Page no
Introduction	3
Project Overview	3
Objectives	4
System Design	4-6
Database Design	7
Implementation	8-12
Result	12-15
Conclusion	16

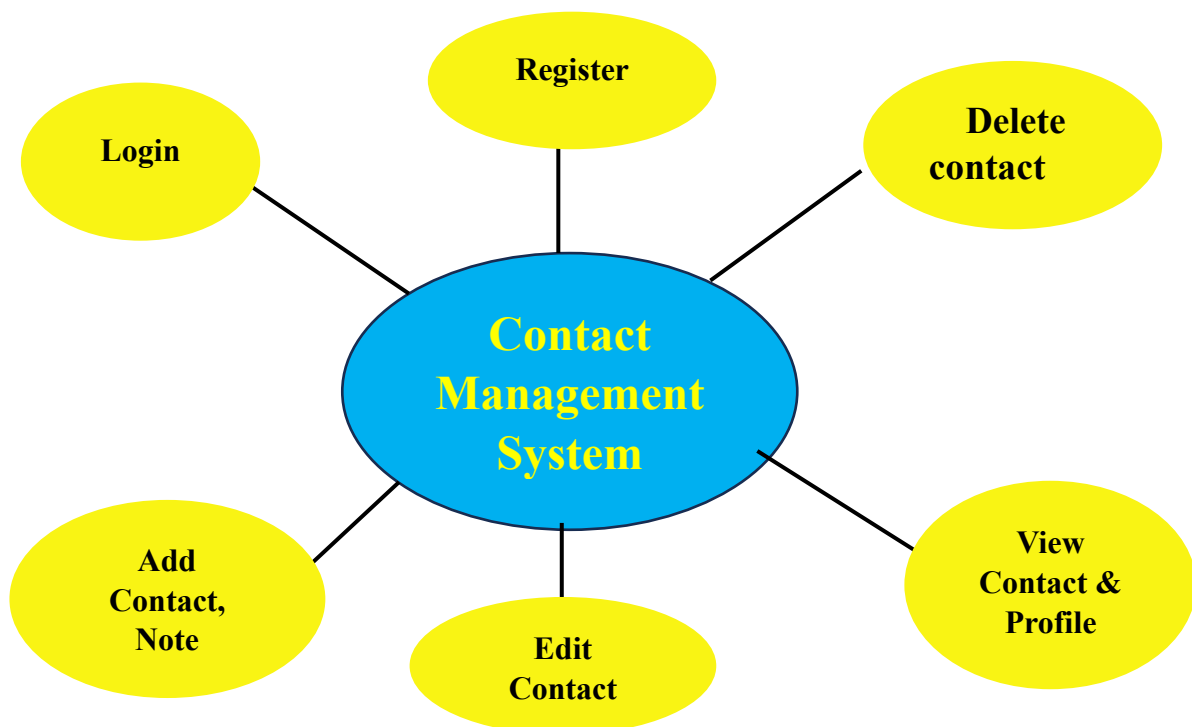
INTRODUCTION

The Contact Management System is a web-based application developed using the MERN Stack (MongoDB, Express.js, React.js, and Node.js) that enables users to manage their contacts digitally. The application provides a secure environment through JWT-based authentication and allows users to perform Create, Read, Update, and Delete (CRUD) operations on contact records. The system also supports adding notes to contacts and offers a responsive user interface for seamless access across multiple devices.

The project was developed during the internship period to gain practical exposure to full-stack web development, API integration, authentication mechanisms, database management, deployment processes, testing methodologies, and software engineering practices.

PROJECT OVERVIEW

The application allows users to:



Objectives of the Project

- Develop a full-stack web application using MERN Stack.
- Implement secure authentication mechanisms.
- Design an efficient database structure.
- Support adding notes to contacts.
- Ensure proper validation and error handling.
- Test application performance and reliability.

SYSTEM DESIGN

System design defines the architecture, components, modules, interfaces, and data flow of an application. It provides a blueprint for implementation.

The Contact Management System follows a client-server architecture using the MERN Stack and adopts the Model-View-Controller design pattern to achieve modularity, maintainability, and scalability.

System Architecture

The Contact Management System follows a three-tier architecture consisting of:

Presentation Layer

- User Interface Components
- Form Validation
- API Requests

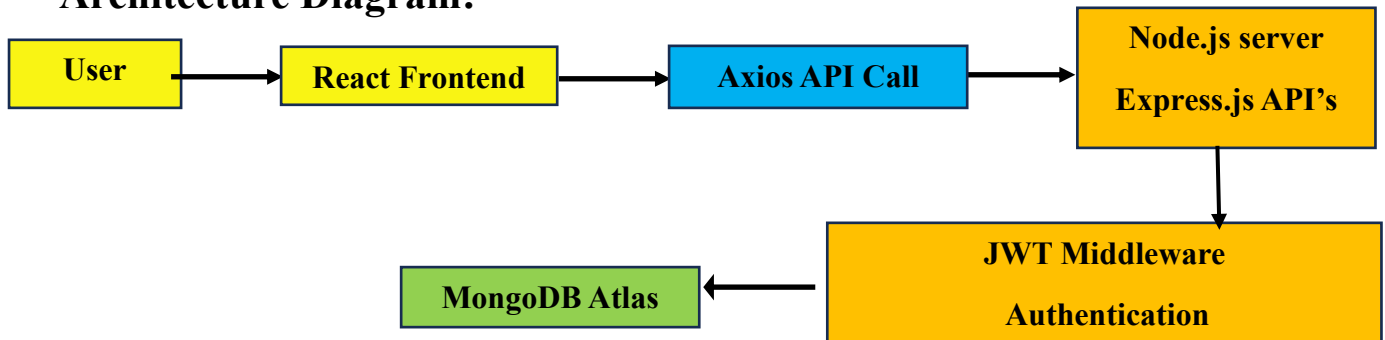
Application Layer

- Node.js
- Express.js, Middle ware
- Authentication Logic

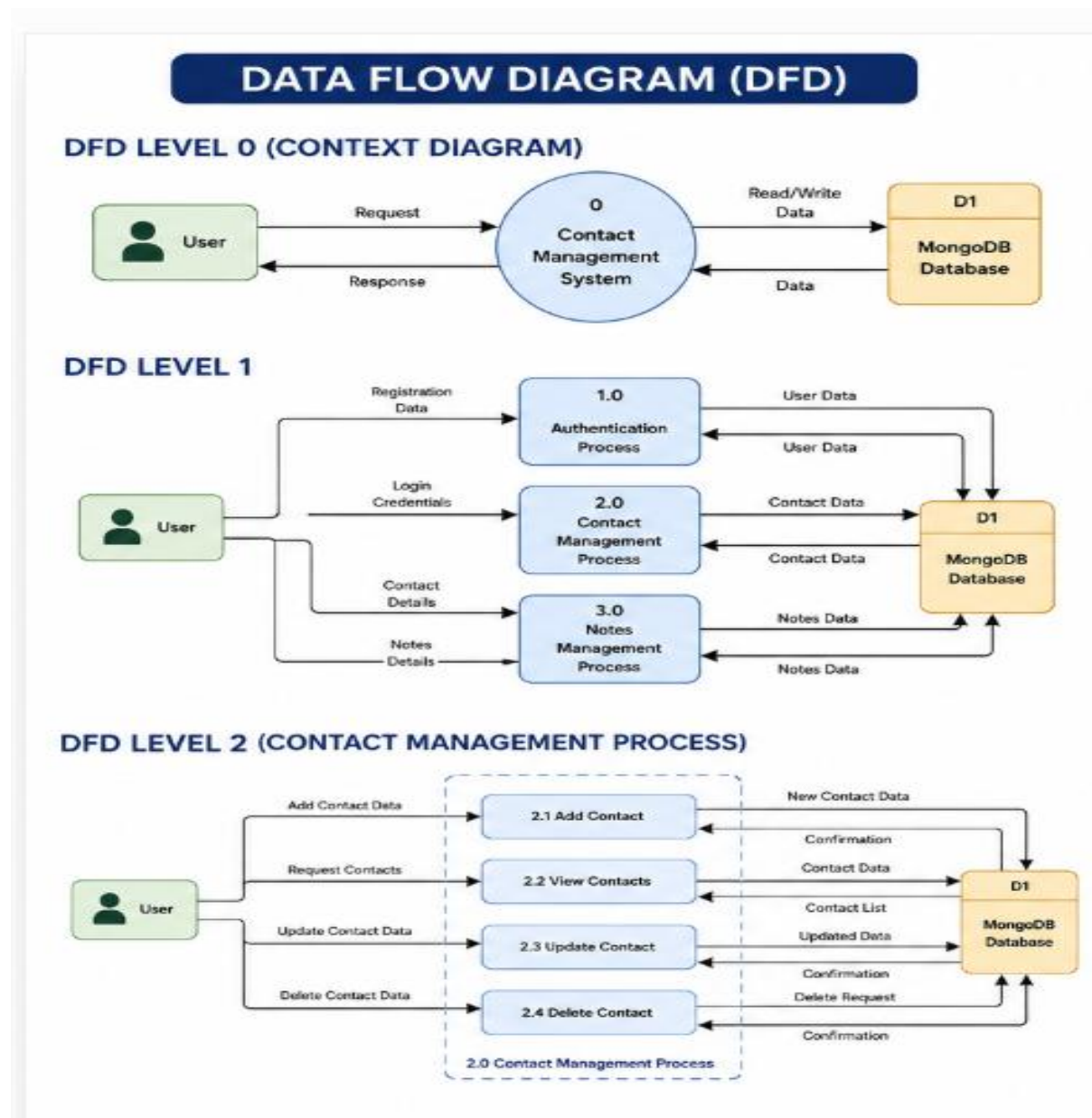
Data Layer

- MongoDB Atlas
- User Collection
- Contact Collection

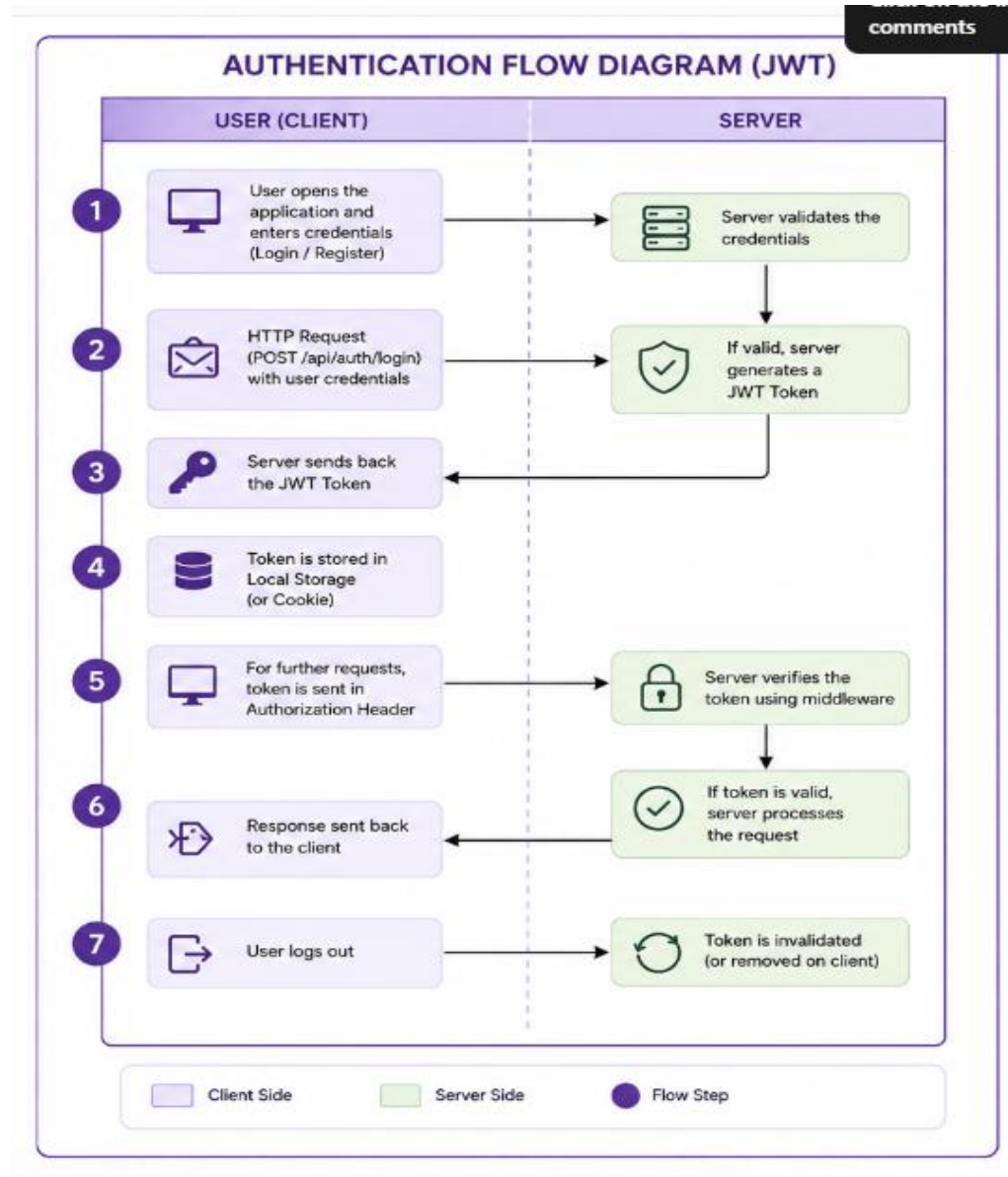
Architecture Diagram:



Data Flow Diagram (DFD)



Authentication:



System Design Summary

The Contact Management System is designed using modern software engineering principles, including MVC architecture, RESTful APIs, JWT-based security, and cloud database integration. The design ensures scalability, maintainability, performance, and secure data management.

DATABASE DESIGN

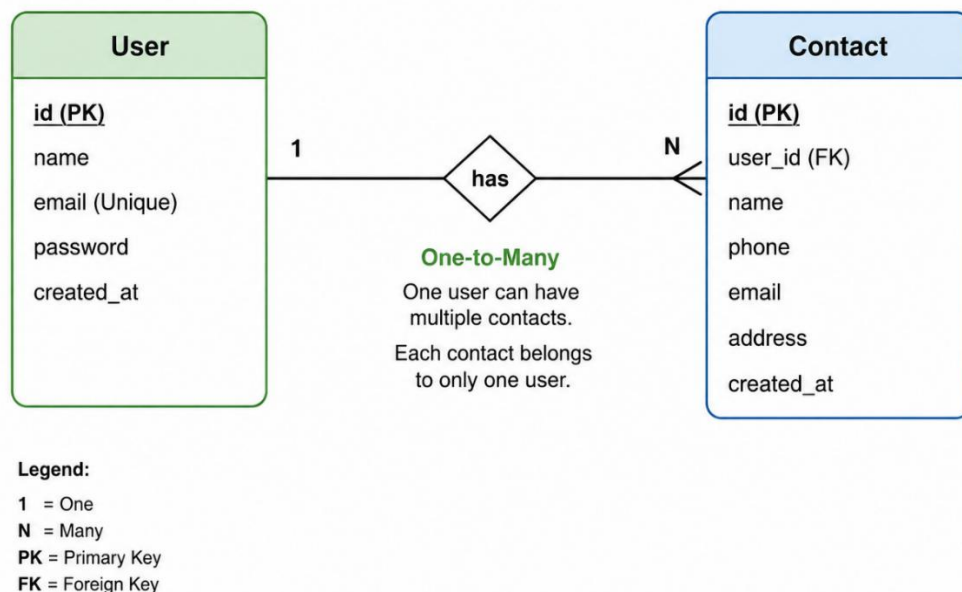
Database design plays an important role in storing and managing application data efficiently. The Contact Management System uses MongoDB Atlas as a cloud-based NoSQL database to store user and contact information securely. MongoDB provides flexibility, scalability, and seamless integration with the MERN stack.

Database Relationship

The database follows a One-to-Many relationship.

- One user can have multiple contacts.
- Each contact belongs to only one user.

ER DIAGRAM



Security Measures

To ensure data security, the following measures were implemented:

- Passwords are encrypted using bcrypt.
- JWT authentication is used for secure access.
- Environment variables are used to secure database credentials.

IMPLEMENTATION

Implementation is the phase where the system design is converted into a working software application. The Contact Management System was implemented using the MERN Stack architecture, providing secure authentication and complete contact management functionality.

Frontend Implementation

The frontend was developed using React.js and Vite to provide a responsive and user-friendly interface.

Main Pages Developed

- Login Page
- Registration Page
- Dashboard
- Add Contact Page
- Edit Contact Page
- Contact List Page

Frontend Features

- Responsive Design
- Form Validation
- Pagination
- API Integration

Backend Implementation

The backend was developed using Node.js and Express.js.

Backend Responsibilities

- Authentication Management
- Contact Management

- Database Operations
- API Development
- Error Handling

Project Structure

config/
controller/
middleware/
models/
routes/
testing/

Authentication Module

Authentication was implemented using JWT and bcrypt.

Registration Process

1. User enters registration details.
2. Password is encrypted using bcrypt.
3. User data is stored in MongoDB.

Login Process

1. User enters credentials.
2. Credentials are validated.
3. JWT token is generated.
4. Token is returned to the client.

Security Features

- Password Hashing
- JWT Authentication

API Integration

Axios was used to connect the frontend with backend APIs.

APIs Used

Method	Endpoint	Purpose
POST	/register	Register User
POST	/login	User Login
GET	/contacts	Retrieve Contacts
POST	/contacts	Add Contact
PUT	/contacts/:id	Update Contact
DELETE	/contacts/:id	Delete Contact

Performance testing was performed on the Contact Management System using K6 to evaluate the reliability and responsiveness of the Login API under concurrent user load. The deployed login endpoint was tested with 10 and 50 virtual users, simulating multiple users accessing the system simultaneously.

The results showed a 100% success rate with no request failures in either test scenario, indicating that the authentication module handled concurrent requests successfully. Although response times increased as the number of users grew, the application remained stable and available throughout the testing process.

Overall, the testing confirmed that the Contact Management System can support multiple simultaneous login requests while maintaining reliable performance. The results also highlight opportunities for further optimization, such as improving API response times, database efficiency, and server scalability.

```

TOTAL RESULTS

checks_total.....: 13      0.301461/s
checks_succeeded...: 100.00% 13 out of 13
checks_failed.....: 0.00%  0 out of 13

✓ login success

HTTP
http_req_duration.....: avg=29.08s min=13.41s med=30.15s max=40.6s p(90)=39.67s p(95)=40.59s
{ expected_response:true }...: avg=29.08s min=13.41s med=30.15s max=40.6s p(90)=39.67s p(95)=40.59s
http_req_failed.....: 0.00% 0 out of 13
http_reqs.....: 13      0.301461/s

EXECUTION
iteration_duration.....: avg=29.14s min=13.41s med=30.23s max=40.69s p(90)=39.76s p(95)=40.67s
iterations.....: 13      0.301461/s
vus.....: 1      min=1      max=10
vus_max.....: 10     min=10     max=10

NETWORK
data_received.....: 51 kB 1.2 kB/s
data_sent.....: 19 kB 450 B/s

running (0m43.1s), 00/10 VUs, 13 complete and 0 interrupted iterations
default ✓ [=====] 10 VUs 30s

C:\Users\Dell\OneDrive\Desktop\contact-mng>
```

```

TOTAL RESULTS

checks_total.....: 35      0.583259/s
checks_succeeded...: 100.00% 35 out of 35
checks_failed.....: 0.00%  0 out of 35

✓ login success

HTTP
http_req_duration.....: avg=32.04s min=6.88s med=32.13s max=59.77s p(90)=54.14s p(95)=55.03s
{ expected_response:true }...: avg=32.04s min=6.88s med=32.13s max=59.77s p(90)=54.14s p(95)=55.03s
http_req_failed.....: 0.00% 0 out of 35
http_reqs.....: 35      0.583259/s

EXECUTION
iteration_duration.....: avg=32.2s min=7.01s med=32.3s max=59.95s p(90)=54.32s p(95)=55.2s
iterations.....: 35      0.583259/s
vus.....: 31      min=31      max=50
vus_max.....: 50     min=50     max=50

NETWORK
data_received.....: 237 kB 4.0 kB/s
data_sent.....: 98 kB 1.6 kB/s

running (1m00.0s), 00/50 VUs, 35 complete and 31 interrupted iterations
default ✓ [=====] 50 VUs 30s

C:\Users\Dell\OneDrive\Desktop\contact-mng>
```

Deployment

The frontend application was deployed using Vercel.

Deployment Steps

1. Build React application.
2. Push source code to GitHub.
3. Connect GitHub repository with Vercel.

4. Configure environment variables.
5. Deploy application.

Deployment Link

Project_Link

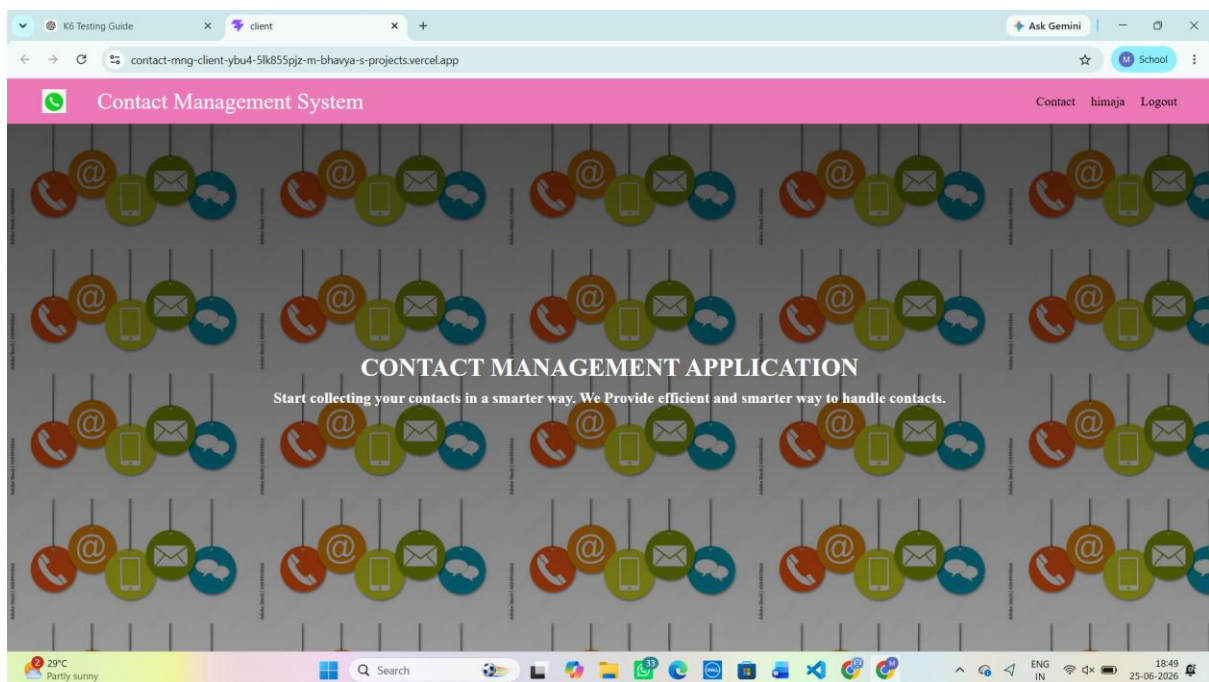
GitHub Link:

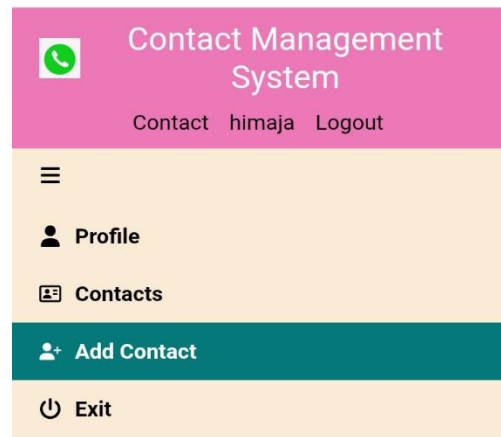
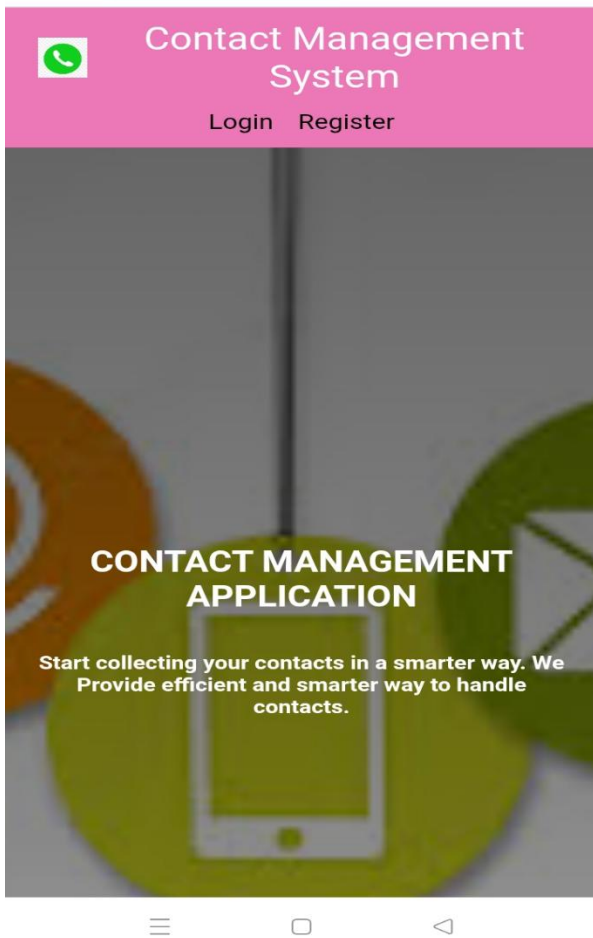
Client - <https://github.com/Bhavya3096/contact-mng-client>

Server- <https://github.com/Bhavya3096/contact-mng-server>

RESULTS :+

Home Page

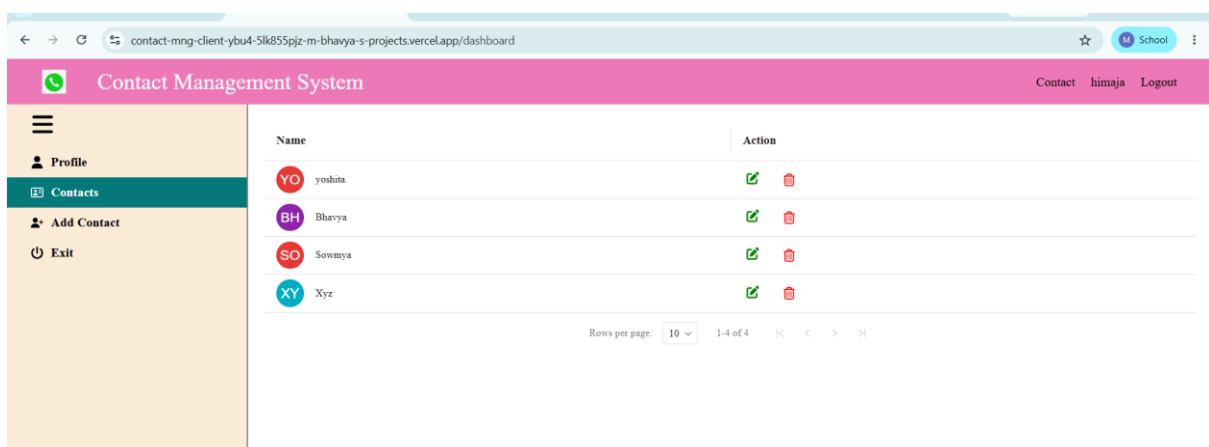




Name	Action
YO yoshita	
BH Bhavya	
SO Sowmya	
XY Xyz	

10

Mobile Layout



Contact Page

Contact Details

Name: Bhavya

Email: bhavya@gmail.com

Phone: 7780341445

Address: 16-3-40, Gandhinagar, vijayawada

Close

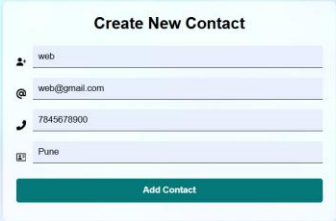
Notes

Add note...

Add Note

No notes added

Contact Card



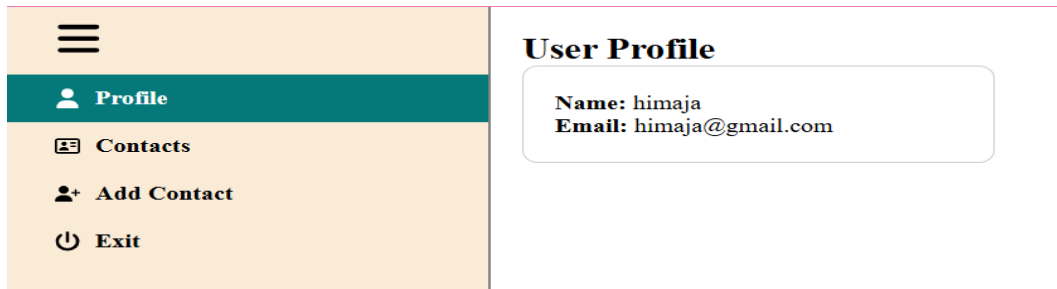
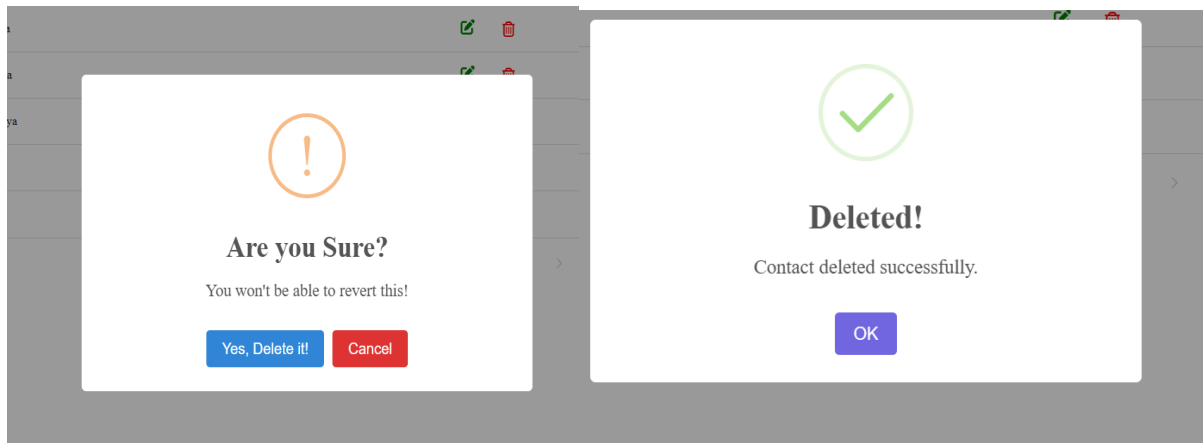
The image shows a 'Create New Contact' form overlaid on a blurred background. The form has a title 'Create New Contact' and five input fields with icons: a person icon for 'Name' (containing 'web'), an '@' icon for 'Email' (containing 'web@gmail.com'), a phone icon for 'Phone' (containing '7845678900'), and a location pin icon for 'Address' (containing 'Pune'). At the bottom is a green 'Add Contact' button.

Add Contact



The image shows an 'Edit Existing Contact' form overlaid on a blurred background. The form has a title 'Edit Existing Contact' and five input fields with icons: a person icon for 'Name' (containing 'Bhavya'), an '@' icon for 'Email' (containing 'bhavya@gmail.com'), a phone icon for 'Phone' (containing '7780341445'), and a location pin icon for 'Address' (containing '16-3-40, Gandhinagar, vijayawada'). At the bottom is a green 'Update Contact' button.

Edit Contact



User Profile

FUTURE ENHANCEMENTS

Although the application fulfills its current objectives, several enhancements can be implemented in future versions.

Proposed Enhancements

- Contact Search Functionality
- Contact Categories and Tags
- Export Contacts to CSV/PDF
- Email Notifications
- Dark Mode Interface
- Contact Sharing Features
- Advanced Dashboard Analytics

These enhancements would improve usability, scalability, and user experience.

CONCLUSION:

The Contact Management System was successfully designed, developed, tested, and deployed using the MERN Stack. The application provides a secure platform for managing contacts through JWT-based authentication and complete CRUD functionality.

The project enabled practical implementation of React.js, Node.js, Express.js, MongoDB Atlas, API integration, authentication mechanisms, performance testing, and deployment. It also provided valuable experience in software design, debugging, testing, and system development practices.

Overall, the internship project successfully achieved its objectives and strengthened both technical and professional skills required for modern full-stack web development.

REFERENCES

1. React.js Official Documentation – <https://react.dev>
2. Node.js Official Documentation – <https://nodejs.org>
3. Express.js Documentation – <https://expressjs.com>
4. MongoDB Documentation – <https://www.mongodb.com/docs>
5. Vercel Documentation – <https://vercel.com/docs>